

PDF extra – extra PDF features for OpTeX

Version 0.3, 2022-12-03

Michal Vlasák, 2021

PDFextra is a third party package for OpTeX. It aims to provide access to more advanced PDF features, which are currently not supported in OpTeX – especially interactive and multimedia features. The development is hosted at <https://github.com/vlasakm/pdfextra>.

In the spirit of OpTeX you may use these macros in any form you want. Either by installing this package and doing `\load[pdfextra]` in OpTeX, or just by copying some useful parts of this package into your documents / packages. OpTeX namespacing is used, but it can be easily stripped, if you wish to incorporate these macros into other macro packages. The code currently depends on LuaTeX, but mostly uses only pdfTeX primitives and a few simple macros from OpTeX. Additionally, the package can be used in plain LuaTeX and LuaLaTeX (in a limited form), see section ??.

User documentation (`pdfextra-doc.tex`) and technical documentation interleaved with source code (`pdfextra.opm`) are all typeset in this PDF file. Some examples of usage are in the user documentation, but file `pdfextra-example.tex` contains more examples.

Contents

Chapter 1

User documentation

1.1 Defining files

Many commands provided by this package require you to supply a file $\langle name \rangle$. This is because many commands either work directly (like inserting attachments or multimedia) or can optionally use files (like inserting JavaScript). The “right” way to use $\langle name \rangle$ is to first define the $\langle name \rangle$ with:

```
\filedef/<type>[<name>]{<path or URL>}
```

Where $\langle name \rangle$ is the name you will use to refer to this file. It is currently limited to ASCII only (as all “ $\langle name \rangle$ s” required by this package). Interpretation of $\langle path or URL \rangle$ depends on the type, which may be:

- e, “embedded file”. The file with path $\langle path \rangle$ will be embedded to the PDF file. A file that is embedded this once, can later be used many times in different contexts, e.g. you may use it to attach a video as an attachment but also have it play on page 1 and even other pages. This is the best way, because the resulting PDF file is self contained.
- x, “external file”. $\langle path \rangle$ can only be a path to the current directory. To refer to the file only $\langle path \rangle$ is used, sort of like a reference. This way the file you want to refer to *has* to be present in the same directory as the PDF file when it is *viewed*!
- u, “URL file”. $\langle URL \rangle$ is the URL of the file you want to refer to.

All these create the same type of object, which ideally could be used interchangeably everywhere a *file specification* is required in PDF. This is sadly not always true. The limitations to only certain types of `\filedef`’s will be mentioned in due sections. But as a rule of thumb, most of the time you want to embed the files into PDF. The external/URL references are good for referring to external files, although other methods are also possible there.

Because most of the times you want $\langle name \rangle$ to be embedded file, you may omit the prior definition and instead use the $\langle path \rangle$ itself. The file will be autoembedded.

Examples:

```
\filedef/u[doc-internet]{http://petr.olsak.net/ftp/olsak/optex/optex-doc.pdf}
\filedef/x[doc-local]{optex-doc.pdf}
\filedef/e[doc-embedded]{optex-doc.pdf}
```

1.2 Multimedia

It is possible to insert video, audio and 3D files for playback/display inside a PDF file (on a page). There are several different PDF mechanisms for inserting multimedia. For audio/video this package uses the so called “Renditions”, which currently have the best support in PDF viewers (fully works in Acrobat and Foxit, partly in Evince and Okular). Rich Media annotations are used for 3D art (works only in Acrobat Reader), although it is possible to also insert audio/video using this mechanism it is very restricting and Renditions are recommended.

Use command `\render[<name>][<optional key-value parameters>]{<appearance>}` for inserting audio/video with Renditions or `\RM[<name>][<optional key-value parameters>]{<appearance>}` for inserting audio/video/3D as Rich Media annotation. The result of this command is similar to what `\inspic` produces¹. Both commands expect $\langle name \rangle$ to be `\filedef`’d name of the file to play/display. As usual, fallback for interpreting $\langle name \rangle$ as path (and embedding it) is in place. It is recommended to only use embedded files with both mechanisms (Rich Media requires it, Renditions with other than embedded files do not work in Acrobat). Optional key-value parameters may be used to customize default values. They may be left out entirely (including brackets). Last parameter, $\langle appearance \rangle$, defines so called “normal appearance”, which is shown before the annotation is activated (audio/video starts playing or 3D scene is displayed). The dimensions of resulting multimedia annotation will be taken from $\langle appearance \rangle$. Most likely you want to use a “poster” picture (inserted with `\inspic`) as appearance – the dimensions will be taken from it and e.g. aspect ratio will be nicely preserved.

¹ But because annotations are involved, transformations using PDF literals will not work as expected.

Table 1.2.1 Key value parameters available for Renditions (`\render`)

Key	Possible values	Default	Description
controls	true or false	false	Whether to display audio/video player controls.
volume	decimal between 0 and 100	100	Audio volume.
repeat	integer ≥ 0	1	Number of repetitions (0 means loop forever).
background	OpTeX color	<code>\White</code>	Color used for part of the annotation not covered by video player (for wrong aspect ratios).
opacity	decimal between 0 and 1	1	Opacity of background .
aactions	<code>\renditionautoplay</code>	(none)	Can be used for autoplay on page open.
name	ascii string	$\langle name \rangle$	Name for use with actions and scripts.

Table 1.2.2 Key value parameters available for Rich Media (`\RM`)

Key	Possible values	Default	Description
activation	explicit or auto	explicit	Whether to automatically activate the annotation on page open or display normal appearance until user clicks.
deactivation	explicit or auto	explicit	Whether to automatically deactivate the annotation on page close or require explicit deactivation by user (from right click menu).
toolbar	true or false	true	Whether to show 3D toolbar (with view and other options).
views	comma separated list of view $\langle name \rangle$ s	$\langle name \rangle$	List of names of 3D views to be used. The default is to try a view of same $\langle name \rangle$. Beware that unknown views are silently ignored.
scripts	comma separated list of script $\langle name \rangle$ s	(none)	List of names of JavaScript script file $\langle name \rangle$ s to be used.
name	ascii string	$\langle name \rangle$	Name for use with actions and scripts.

Customization of Renditions is possible using key-value parameters, but beware that it mostly doesn't work in Evince and Okular (Acrobat and Foxit are fine in this regard). The available parameters are in Table ???. Most customizations of Rich Media concern 3D art. Available parameters are listed in Table ???.

The weird `name` key is only required if one media file is used more than once and control using actions or JavaScript scripts is needed.

Examples of video insertion:

```
% embed file under name "video"
\filedef/e[video]{example-movie.mp4}
% insert video into page using Renditions mechanism with controls and autoplay
\render[video][
  name=bigvideo,
  controls=true,
  aactions=\renditionautoplay,
]{\picwidth=\hsize \inspic{example-image.pdf}}

% render the same file again, but with different dimensions, no controls
% and explicit activation
\render[video]{\inspic{example-image.pdf}}
```

When displaying 3D there are more things involved. First, only U3D and PRC can be included in PDF files. The simplest way to show the 3D scene on page is without any optional parameters:

```
\RM[part.prc]{\picwidth=\hsize \inspic{part.png}}
```

The resulting view will be what is defined in the 3D file. But it is possible to customize it, by creating custom view. Or even more of them – they will be available in the user interface for easy switching, first one is considered default. Parameters not defined in a custom view often take what is in the 3D file as default value. `\DDDview[⟨view name⟩][⟨key-value parameters⟩]` is the command for defining 3D views. The brackets surrounding key-value parameters have to be included even if no key-value parameters are used. The available parameters are explained in Table ??.

Table 1.2.3 Key value parameters available for 3D views (`\DDDview`)

Key	Possible values	Default	Description
projection	perspective or ortho	perspective	Projection type to use (perspective distorts the view, e.g. parallel lines are not shown as such, but is more natural to human eye, orthogonal projection is what is generally used with technical parts).
scale	decimal number	1	Scaling to use when orthogonal projection is used.
FOV	number between 0 and 180	30	Field of view for use with perspective projection.
background	OpTeX color	<code>\White</code>	Color used as background in the 3D scene.
rendermode	Solid, SolidWireframe, Transparent, TransparentWireframe, BoundingBox, TransparentBoundingBox, Wireframe, ShadedWireframe, Vertices, ShadedVertices, Illustration, SolidOutline, ShadedIllustration	(taken from 3D file)	Used rendering mode. See PDF standard ⁱ for more details.
lighting	White, Day, Night, Hard, Primary, Blue, Red, Cube, CAD, Headlamp	(taken from 3D file)	Used lighting scheme. See PDF standard ⁱⁱ for more details.
method	media9, manual, u3d	media9	Method used for defining 3D camera position and orientation.

ⁱhttps://www.adobe.com/content/dam/acom/en/devnet/pdf/pdfs/PDF32000_2008.pdf#G12.2358303

ⁱⁱhttps://www.adobe.com/content/dam/acom/en/devnet/pdf/pdfs/PDF32000_2008.pdf#G12.2358356

The value of `method` key influences what other key-value parameters are available. For details about the manual method see the technical documentation (??). The `u3d` method works only with U3D files (not with PRC files) and requires you to know the internal “path” of the view contained in the file and is hence not always useful (especially when the paths are weirdly constructed by exporting applications). But if you know the path you can use “`method=u3d, u3dpath=⟨path⟩`”. All Unicode characters are allowed in `⟨path⟩`.

Table 1.2.4 Key value parameters available for media9 method of 3D views (`\DDDview`)

Key	Possible values	Default	Description
<code>coo</code>	three space separated (decimal) numbers	0 0 0	“Center of orbit”. Coordinates of the point camera is supposed to look at.
<code>roo</code>	(decimal) number	0	Distance of camera from <code>coo</code> .
<code>c2c</code>	three space separated (decimal) numbers	0 -1 0	“Center of orbit to camera” vector. A directional vector (i.e. length doesn’t matter). The direction the camera will be looking at is opposite of this vector. Default is view towards positive y .

The most useful method of specifying the camera position/orientation is `method=media9`. The same method is also used in a number of other packages (`movie15`, `rmannot`, `ConTeXt`). Its input are key-value parameters, listed in Table ?? . For illustration of the parameters check the documentation² of `media9`.

You can construct simple views only by using these few parameters. When talking about meanings/names of different views it is needed to know the real placement of the 3D object in the 3D world. But for a reasonably orientated object in the center (`coo=0 0 0`) constructing simple views is very easy (keep in mind `method=media9` is default):

```
\DDDview[front][
  projection=ortho,
  roo=400,
]
\DDDview[left][
  projection=ortho,
  roo=400,
  c2c=-1 0 0,
]
\DDDview[top][
  projection=ortho,
  roo=400,
  c2c=0 0 1,
]
\DDDview[isometric][
  projection=ortho,
  roo=500,
  c2c=-1 -1 1,
]
```

We can then perhaps use these views in `\RM`:

```
% Auto activated 3D Rich Media annotation. White background just ensures
% the right dimensions. Comma in `views` is escaped using "{ }".
\RM[part.prc][
  activation=auto,
  views={front, left},
]{\White\vrule width\hsize height\vsizex}

\RM[part.prc][ % the same 3D file, now with different views
  name=part2,
  activation=auto,
  views={top, isometric},
]{\White\vrule width\hsize height\vsizex}
```

² <https://mirrors.ctan.org/macros/latex/contrib/media9/doc/media9.pdf#figure.8>

If you want to deduce the view parameters automatically it is possible. You can just not specify any view, but include the `3Dmenu.js` script from `media9` package. It enables you to right click the annotation and select “Get Current View” (or even “Generate Default View” which finds the view all by itself). A window with generated parameters will show up. You can then copy the ones this package understands (`c2c`, `coo`, `roo`) and use them. You don’t have to be excessive with precision, because after calculation everything gets rounded to 6 decimal places anyways.

```
% using 3Dmenu.js to generate 3D view parameters automatically
\RM[part.prc] [
  name=part3,
  activation=auto,
  scripts=3Dmenu.js,
]{\White\vrule width\hsize height\vsiz}}
```

The use of `scripts` isn’t limited to this though, there are many other possibilities. First, you can use as many scripts as you want (`scripts={script1.js, script2.js, ...}`), but be careful that 3D JavaScript is kind of special, and has to come from embedded files (here we were using the auto-embedding, but we could have used e.g. `\filedef/e[3dmenu]{3Dmenu.js}` and then “3dmenu” instead). For creating your own scripts check out the Acrobat 3D JavaScript API³. It is possible to do different transformations and achieve animations using “time events”. The implicit “context” of 3D scripts can be accessed from normal JavaScript actions (see ??). Vice versa 3D scripts may access the global JavaScript environment using `host` object.

More examples of 3D Rich Media, including usage of 3D JavaScript, are available in the example file `pdfextra-example.tex`. They show how it is possible to port the examples used by `media9`.

1.3 Actions

Actions are very important aspect of interactivity in the context of PDF. There are a few very useful types of actions, like “goto” actions which jump to other part of document. There are also a few ways how to *execute* actions. The most usual is a clickable area on page, but clickable bookmarks (“document outline”) also execute actions behind the scenes. OpTeX supports only basic “goto” and “URI” actions using `\ilink` (used for `\ref`, `\cite`, etc.) and `\ulink` (used for `\url`).

This package offers generalization of this mechanism. The core of it is a way of specifying an action. This syntax is called $\langle action\ spec \rangle$ and is used for example by `\hlink` command, which can replace both `\ilink` and `\ulink`. $\langle action\ spec \rangle$ is a comma separated list of $\langle type \rangle : \langle arguments \rangle$. where $\langle type \rangle$ refers to the action type and the syntax of arguments is dependant on $\langle type \rangle$. Leading spaces are ignored, trailing aren’t. You probably won’t often use the possibility of specifying multiple actions, but it is for chaining execution of several actions.

`\pdfaction[ $\langle action\ spec \rangle$ ]` is available for lower level creation of different actions, but for clickable areas on page you will use `\hlink[ $\langle action\ spec \rangle$ ]{ $\langle text \rangle$ }` with a very similiar syntax. Although note, that `\hlink`’s interface is also not really high level and wrapping it inside macros like `\ref` or `\url` might be beneficial. $\langle text \rangle$ will be typeset directly and the area it occupies will be clickable. Clicking it executes action defined by $\langle action\ spec \rangle$. Line breaks inside $\langle text \rangle$ will be possible, in that case several clickable rectangles will be created, one for each line. Normally in text you want the areas to be of the same height and depth (calculated from `\baselineskip`), to achieve sort of a lining, uniform effect. If you want to define big clickable buttons, you may need to turn off the lining effect using `\nolininglinks`. It respects groups, but a counterpart (`\lininglinks`) is also available.

There are a few predefined action types ($\langle type \rangle$): `url`, `extref`, `extpgref`, `named`, `transition`, `js`, `goto3dview` and `rendition`. They will be explained in a moment. Any unrecognized $\langle type \rangle$ is understood as an “internal link”, where $\langle type \rangle : \langle link \rangle$ is the destination of the link. Hence it is possible to use `\hlink` as `\ilink` for example with OpTeX’s normal types of internal links. For example:

See `section~\hlink[ref:section]{2.2.13}` or `page~\hlink[pg:5]{5}`.

`\ulink` may be replaced like this:

Visit CTAN’s `\hlink[url:https://www.ctan.org/]{website}`.

Before we really get into different types of actions, there is a nicer command for setting the initial (“open”) action of PDF document, which is executed when the document is opened. It defaults to

³ https://wwwimages.adobe.com/content/dam/acom/en/devnet/acrobat/pdfs/AcrobatDC_js_3d_api_reference.pdf

opening the page with the zoom level according to viewing user's preferences. But you can change this using `\openaction[<action spec>]`:

```
\openaction[pg:2]
```

1.3.1 External references

There are two actions analogous to internal links that can be used to link to external documents. `[extref:<name>:<named destination>]` can be used to refer to named locations in a PDF document prepared by `\filedef` with *<name>*. `[extpgref:<name>:<page number>]` is similar but refers to a page number. Although it would be nice, these actions aren't well supported by all viewers.

Example:

```
\hlink[extref:doc-internet:ref:langphrases]{0pTeX documentation,
    section \"Multilingual phrases and quotation marks\".}
\hlink[extpgref:doc-internet:12]{0pTeX documentation, page 12.}
```

(Note that “:” in “`ref:langphrase`” is not part of the syntactic rule, it is just the value of *<named destination>* in this case.)

A little bit of customization is possible, see ??.

Because of the poor support you may find luck with the less universal url action with `#fragment`:

```
\hlink[url:http://petr.olsak.net/ftp/olsak/optex/optex-doc.pdf#ref:langphrases]
    {0pTeX documentation, section \"Multilingual phrases and quotation marks\".}
```

1.3.2 Named actions

There are four defined in PDF standard, but viewers may support more. All in examples:

```
\hlink[named:NextPage]{Go to next page,}
\hlink[named:PrevPage]{go to previous page,}
\hlink[named:FirstPage]{go to first page,}
\hlink[named:LastPage]{go to last page.}
```

1.3.3 Transition actions

Transition actions generally make sense only when chained *after* jump actions. The specified transition/animation will occur, before destination is opened, but it will not override a transition defined for the particular page. The syntax is `[transition:<transition spec>]`. See ?? for more information about *<transition spec>*.

Example:

```
\hlink[ref:yellow-slide, transition:Box:3:/M /O]
    {Go to yellow slide with long outward Box transition}
```

1.3.4 JavaScript actions

They allow executing pieces of JavaScript code using syntax: `[js:<name or script>]`. If *<name or script>* is a validly `\filedef`'d the script from file *<name>* will be executed (only embedded files are valid). Otherwise *<script>* will be used directly. Apart from reuse in different documents, scripts loaded from files may be encoded in UTF-16BE and hence support Unicode, inline *<script>*'s current don't.

Examples:

```
\openaction[js:{%
    app.alert("Javascript alert, open action");
    console.println("printing to console from openaction");
}]

\filedef/e[jstest]{test.js}
\hlink[js:jstest]{JavaScript action from file}
```

(Note how braces were used to guard the comma in JavaScript code from being interpreted as a action separator.)

In these actions you may want to use your own functions which should be defined before user has a chance of activating any other JavaScript actions. This is the purpose of “document level”

JavaScript actions. They function exactly the same way as normal JavaScript actions, but have names (although meaningless, they must be unique) and are executed in order of definition. Use `\dljavascript[⟨name⟩]{⟨name or script⟩}` for defining these actions:

```
\filedef/e[preamble]{preamble.js}
\dljavascript[preamble]{preamble}

\dljavascript[initialization]{%
  var data = 42;
  function getRandomNumber() {
    return 4; // chosen by fair dice roll, https://xkcd.com/221/
  }
  console.println("initialized with seed " + getRandomNumber());
}
```

The JavaScript API available is not the same as the one in the web browsers. It is instead specified by Adobe: https://www.images.adobe.com/content/dam/acorn/en/devnet/acrobat/pdfs/js_api_reference.pdf.

1.3.5 3D JavaScript actions

There are no “special” JavaScript actions for dealing with 3D. Normal JavaScript actions are used. You just have to access the 3D context of the annotation you want to control. For annotation with `⟨name⟩` the context is made available in `\DDDcontext{⟨name⟩}`. Under this context object you find all global definitions from the respective 3D scripts. For example if a “turn” function is defined, it is possible to call it like this:

```
\hlink[js:\DDDcontext{part3}.turn();]{Turn by 90 degrees along $$ axis.}
```

1.3.6 GoTo3Dview actions

GoTo3Dview actions allow changing the view of the 3D scene to one of the predefined views (those listed by `views` comma separated list of `\RM`). The syntax is `[goto3dview:⟨name⟩:⟨view⟩]`. `⟨name⟩` is name of the annotation whose view we want to change and `⟨view⟩` is the intended view. You can refer to last inserted Rich Media annotation using empty name. For `⟨view⟩` you can either use “(⟨view name⟩)” (e.g. “(part)”), index of the view in the view list (zero based, e.g. “0” for the first view) or one of the special values: “/N” (next), “/P” (previous), “/F” (first) “/L” (“last”).

```
% let's define an annotation with a few views
\RM[part.prc] [
  activation=auto,
  views={front, left, top, isometric},
]{\White\vrule width\hsize height\vsizex}

% try the different methods of reffering to views
\hlink[goto3dview:/N]{Next view},
\hlink[goto3dview:(left)]{left view} and
\hlink[goto3dview:part.prc:3]{third view}.
```

1.3.7 Rendition actions

Rendition actions can be used to control playback of “Rendition annotations”. They use the syntax `[rendition:⟨name⟩:⟨operation⟩]`, where `⟨name⟩` refers to the name of the rendition to control and `⟨operation⟩` is one of `play`, `stop`, `pause` or `resume`. As a convenience, you can refer to last inserted rendition using empty name. If a file has been `\rendered` more than once with the same name, the action will influence the first instance.

Beware that currently these actions do not work in Evince and Okular (but do in Acrobat and Foxit). Examples:

```
% rendition with name=video, that we want to control
\render[video]{\inspic{example-image.pdf}}

% we want the rendition action to have yellow border and red content
```

```
\def\_renditionborder{1 1 0}
\let\_renditionlinkcolor\Red
```

To start playing the video, click `\hlink[rendition::play]{\"Play\"}`.
After that you can `\hlink[rendition:video]{pause}`.

1.4 Transitions and other page attributes

In PDF there are a few settings that can be set as *page attributes*. This means that they apply only to said page. For setting these page attributes, there are two options:

- value for “current page” (or rather the page where the command appears),
- default value used if “current page” value is not set.

While this package contains mechanism to handle all page attributes, not that many are useful for end user. The interesting ones remaining are:

- Transitions and page durations. When page has the transition attribute any jump to this page will display the requested animation (customized by the corresponding parameters). Page duration is the time before PDF viewer will auto advance to the next page. `\transition[⟨transition spec⟩]` sets transition for the “current page”, `\transitions` sets the default. *⟨transition spec⟩* has three parts:

⟨animation type⟩:⟨duration⟩:⟨raw PDF attributes⟩

⟨animation type⟩ is one of `Split`, `Blinds`, `Box`, `Wipe`, `Dissolve`, `Glitter`, `Fly`, `Push`, `Cover`, `Uncover` and `Fade`, or the special value `R` which essentially means no animation and instantaneous transition (regardless of the set duration of transition). *⟨duration⟩* is the duration of transition in seconds (integer or decimal number). *⟨raw PDF attributes⟩* may be used to customize the animations (for example `/M` can set the direction of motion of `Split`, `Box` and `Fly`, e.g. `/M /I` for inward motion). For the raw PDF attributes refer to the standard itself⁴. *:⟨raw PDF attributes⟩* or even *:⟨duration⟩:⟨raw PDF attributes⟩* may be omitted. Default values specified by PDF standard will be used in that case (1 second duration and default values for all attributes). Page durations can be set either using `\defaultpageduration[⟨duration⟩]` or `\pageduration[⟨duration⟩]`, where duration is in seconds (default is no auto advancement, i.e. ∞). Examples:

```
% unless stated otherwise all pages will have Wipe animation
% with 1 second duration
\transitions[Wipe:1]

% but this page has a 1 second Fade animation
\transition[Fade]

\pg+ % (if using \slides)

% and this page has 3 second Split animation
% with vertical direction and inward motion
\transition[Split:3:/Dm /V /M /I]
```

Note that transitions are only displayed when in *full-screen mode*. You can use `\fullscreen` to have the document automatically open in full-screen mode.

- Additional actions. It is possible to define actions which will respond to page events: page open (`/O`) and page close (`/C`). They can be set using `\defaultpageactions[⟨additional actions spec⟩]` (the default for all pages) and `\pageactions[⟨additional actions spec⟩]` (current page override). *⟨additional actions spec⟩* consists of braced pairs of “event” (`O` or `C` in this case) and *⟨action spec⟩*. Example:

```
\pageactions[
  {O} {js:{app.alert("Page open, random = " + getRandomNumber());}}
  {C} {js:{app.alert("Page close!");}}
]
```

⁴ https://wwwimages2.adobe.com/content/dam/acom/en/devnet/pdf/pdfs/PDF32000_2008.pdf#G11.2295795

1.5 Attachments

Every file embedded into a PDF file may optionally be presented in the user interface as an embedded file. This allows readers of the document to save or open the file.

PDF allows two ways of presenting attachments – using annotations (the attachments is represented by a rectangular icon on a page) or global array of attachments (attachments are visible in PDF viewers toolbar). The first method is intended more for document reviewers than for primary insertion. Hence this package supports only the second type.

You may add an embedded file to the global attachments array using `\attach[⟨name⟩]`. As usual, name is either a `⟨name⟩` defined with `\filedef` or alternatively a path to file which will be embedded (and `\filedef`) automatically. It is not possible to `\attach` files referenced by path or URL.

If you want to automatically display toolbar with embedded files, consider using `\showattached` (see section ??).

1.6 Document view

You can choose what is shown when document is opened with commands:

- `\fullscreen` (the document is opened in full-screen mode),
- `\showoutlines` (show bookmarks/outlines toolbar)
- `\showattached` (show attachments in toolbar)

The commands are mutually exclusive and only the first appearing one will be respected.

You can request two page view (odd pages on the right) using `\duplexdisplay`. It is useful for more natural display of double sided documents. Because it may not be desirable to automatically apply this, it is independent of `\margins`.

1.7 Usage in plain LuaTeX or LuaLaTeX

You can use this package also from plain LuaTeX by adding to your document:

```
\input pdfextra
```

or for LuaLaTeX:

```
\usepackage{pdfextra}
```

See the file `pdfextra-example-latex.tex` for the adaptation of the OpTeX examples to L^AT_EX.

The usage of the macros described in this document is the same, but there are limitations:

- *Color.* Where this package expects “OpTeX color” key value argument (e.g. `\Blue`), you have to use an RGB triplet instead (e.g. `0.0 0.0 0.0` or the shorter `0 0 0`).

But for text color setting, you can get away with wrapping commands from L^AT_EX’s `color` package, e.g. to customize link border/color:

```
\sdef{_renditionlinkcolor}{\color[red]}
```

- *Initialization.* In OpTeX you normally have to initialize hyperlinks with the command `\hyperlinks⟨color for internal links⟩⟨color for external links⟩`.

This is not required by this package. You can instead set the color by setting `\_linkcolor` (fallback for all link types), `\_ilinkcolor` / `\_elinkcolor` (internal / external links). E.g. if you have the L^AT_EX `color` package loaded, you can get blue links like this:

```
\sdef{\_linkcolor}{\color[blue]}
```

In addition to the above this means all links will be blue except “rendition” links.

- *Openaction.* If the package `hyperref` is used, then `\openaction` will not work.
- *Labels / hyperlink destinations.* OpTeX uses very simple and consistent scheme for labels / hyperlink destinations:
 - `ref:⟨label⟩` – result of `\label[⟨label⟩]`
 - `toc:⟨tocrefnum⟩` – result of `\chap/\sec/\secc` titles
 - `pg:⟨gpageno⟩` – created on each page with global numbering from 1
 - `cite:⟨bibpart⟩/⟨bibnum⟩` – bibliography references,
 - `fnt:⟨gfnotenum⟩` – link from text to footnote
 - `fnf:⟨gfnotenum⟩` – link from footnote to text
 - `url:⟨url⟩` – used by `\url` or `\urlink`,

Hence these labels / destinations can be used with `\hlink`, e.g. to make text **page 5** a link to page 5, one can use:

```
\hlink[pg:5]{page 5}
```

This is not possible in plain Lua_T_EX (no destinations are created) or LuaLa_T_EX (different and incompatible destination names). You would have to create your own destinations adhering to the naming convention $\langle type \rangle : \langle arguments \rangle$ to be able to use `\hlink` as intended for some links.

Chapter 2

Technical documentation

This is the technical documentation. It is intended for those who want to know how this package works internally. Casual users shouldn't need to read this. But if you would like to customize anything or perhaps just use some part of this package, feel free to copy paste and use anything you want in OpTeX's spirit.

This documentation is interleaved within the source itself, both are contained in a single file, `pdfextra.opm` (according to OpTeX conventions). The user documentation is instead contained in `pdfextra-doc.tex`, which itself `\input's` the documented source file `pdfextra.opm` so that the user and technical documentation is available in a single PDF file, `pdfextra-doc.pdf`.

```
6 \def\pdfextra_version{0.3}
7 \def\pdfextra_date{2022-12-03}
8 \codedec1 \RM {Extra PDF features <v\pdfextra_version, \pdfextra_date>}
9 \namespace{pdfextra}
```

`pdfextra.opm`

2.1 Package initialization

We ensure that hyperlinking is active. Our fallback `\_linkcolor` must exist. We also use it for `\hyperlinks` if the user didn't enable `\hyperlinks` yet (we don't want to override user setting).

```
19 \ifdefined\_ilinkcolor\_else
20   \ifdefined\_linkcolor
21     \ea\_let\_ea\_linkcolor \ifdefined\Blue\Blue\_else\_empty\_fi
22   \else
23     \let\_linkcolor\linkcolor
24   \fi
25 \fi
26
27 \ifx\_dest\_destactive\_else
28   \hyperlinks\_linkcolor\_linkcolor
29 \fi
```

`pdfextra.opm`

We are in the OpTeX package namespace. A couple of shortcuts are defined here: `\.isdefined`, `\.trycs`, `\.cs \.slet`, `\.slet`, `\.sdef` and `\.sxdef`. They all hard code the package name, because we already have too many levels of indirection.

```
38 \def\.isdefined#1{\isdefined{pdfextra_#1}}
39 \def\.trycs#1{\_trycs{pdfextra_#1}}
40 \def\.cs#1{\_cs{pdfextra_#1}}
41 \def\.slet#1#2{\_slet{pdfextra_#1}{pdfextra_#2}}
42 \def\.sdef#1{\_sdef{pdfextra_#1}}
43 \def\.sxdef#1{\_sxdef{pdfextra_#1}}
```

`pdfextra.opm`

2.2 Helper macros

The macros here are just helpers for the macros to follow. They are not useful generally, but proved useful in the expandable context of writing to PDF files.

Already the first one limits the use to LuaTeX (but who needs other engines anyways :). `\.emptyor` *(possibly empty text)* *(text to use when first argument is nonempty)* checks whether the first argument is empty, if not it expands the second argument which can use the text from the first argument with `\.nonempty`. `\.attroempty` *(attribute name)* *(value)* builds upon the first one and is really useful for PDF dictionaries, when we don't want to write an attribute without a value (a default specified by standard will be used instead).

```
60 \def\.emptyor#1#2{%
61   \immediateassignment\_edef\.nonempty{#1}%
62   \ifx\.nonempty\_empty\_else #2\_fi
63 }
64 \def\.attroempty#1#2{\.emptyor{#2}{/#1 \.nonempty}}
```

`pdfextra.opm`

There is a dilemma for handling colors. While typesetting it is possible to use greyscale, CMYK or RGB colors. But there are contexts where it is possible to only use RGB colors. We want to provide the user with two possibilities of specifying colors:

- RGB color using PDF triplet (e.g. 1 0 0),
- OpTeX color using control sequence (e.g. \Blue)

Both are handled by `\.colortorgbdef<cs><color specification>`, which defines `<cs>` to the corresponding PDF RGB triplet. The indirection with defining a macro is because we want to use the processed color within expansion only contexts where grouping is not possible.

pdfextra.opm

```
83 \_def\.colortorgbdef#1#2{\_bgroup
84   \_def\_setrgbcolor##1{##1}%
85   \_def\_setcmymcolor##1{\_cmymtorgb ##1 ;}%
86   \_def\_setgreycolor##1{##1 ##1 ##1}%
87   \_xdef#1{#2}%
88   \_egroup
89 }
```

`\.xaddto\macro<text>` is a natural extension of OpTeX's `\addto` that expands `<text>` and is global. `\.tmp` is used through the package for temporary values.

pdfextra.opm

```
97 \_def\.xaddto#1#2{\_edef\.tmp{#2}%
98   \_global\_ea\_addto\_ea#1\_eaf\.tmp}%
99 }
```

This package defines a few commands in the form `\macro[<name>][<optional arguments>]{<text>}`. To make it possible to omit the `[<optional arguments>]` `\.secondoptdef` is defined.

`\.secondoptdef<macro><parameters>{<body>}` defines `\macro` with first mandatory argument in brackets (saved to `\.name`). Second optional argument in brackets is scanned using helper macro defined with `\optdef` and is saved to `\_opt` token list). Additional `<parameters>` can be specified as with `\optdef` (numbered from #1).

pdfextra.opm

```
113 \_def\.secondoptdef#1{%
114   \_def#1[##1]{\_def\.name{##1}\_cs{sopt:\_string#1}}%
115   \_ea\_optdef\_csname\_pdfextra_sopt:\_string#1\_endcsname[]%
116 }
```

When processing comma separated lists sometimes it is needed to ignore the remaining text. For this we use `\.untilend` macro which ignores everything up to dummy `\.end`. This is analogous to OpTeX's `\_finbody` used for the same purpose. Sometimes `\.end` is used as sentinel and compared in `\ifx` tests, hence we define it to a unique value.

pdfextra.opm

```
126 \_def\.untilend#1\.end{}
127 \_def\.end{\_pdfextra_end}
```

For various uses it is necessary to know the number of page where something happens. This has to be handled asynchronously with `\write`. Here we use OpTeX specific `.ref` file and associated macros, but this could be replaced as long as the same interface is exposed.

`\.setpageof<name>` writes `\.Xpageof<name>` to the `.ref` file. In the next TeX run `\.Xpageof` finds out the page number (`\_gpageno`) from OpTeX's `\_currpage` and saves it so that `\.pageof<name>` can retrieve it. In the first run we can't be sure of the page where the content will end up. As a rough estimate we take the current page – this actually works well for slides where page breaks are manual.

When `.ref` file is read along with the definition of `\.Xpageof` this package has not been loaded yet. Hence we can't use namespaced variants of `\.isdefined`, etc.

pdfextra.opm

```
147 \_refdecl{%
148   \_def\.Xpageof#1{\_isdefined{\_pdfextra_pageof:#1}\_iffalse
149     \_sxdef{\_pdfextra_pageof:#1}{\_ea\_ignoresecond\_currpage}\_fi
150   }%
151 }
152
153 \_def\.setpageof#1{\_openref \_ewref\.Xpageof{#1}}
154
155 \_def\.pageof#1{%
156   \_trycs{pageof:#1}%
157   \_the\_numexpr\_gpageno+1\_relax % best effort = current page num
158 }%
159 }
```

2.3 Handling of files

Handling of files is a big topic of this package. Files are everywhere – files containing multimedia, JavaScript script files, attachments, externally referred files. . . Therefore a more sophisticated mechanism for handling files is needed. The mechanism introduced in this section handles all three cases of a *file specification*:

- files embedded in the PDF (“e”, embedded file),
- files determined by path (“x”, external file),
- files determined by URL (“u”, url file).

Although ideally all three would be interchangeable this is not always the case, because e.g. some media files must be embedded and linking to external resources does not work with embedded files.

In most cases there are two many names and other associated values involved:

- Some kind of a “friendly” name. This one is sometimes shown by PDF viewers.
- The real name of the file. Also shown but in different contexts.
- The path or URL used to determine the file.
- MIME type of the file.

For example when talking about OpTeX’s documentation we might have a friendly name of “opdoc”, file name of “optex-doc.pdf”, URL of “<http://petr.olsak.net/ftp/olsak/optex/optex-doc.pdf>” and MIME type of “application/pdf”. Different subset of them is required in different contexts, but the user should only have to specify the friendly name (by which they will refer to the file) and the path/URL of the file. The rest will be deduced. The friendly name is used as a handle and *is usable* in all places where file specification is required (although it may not produce conforming output, see above).

In this two step process – definition and (re)use – we introduce a command for defining files: `\filedef<(type)> [<friendly name>]{<path or URL>}`. The macro itself does general definitions and dispatches the type dependant work to other macros in the form `_filedef:<(type)>`.

pdfextra.opm

```
205 \_def\filedef/#1#2[#3]#4{%
206   \sxddef{filename:#3}{(\.filename{#4})}%
207   \_edef\tmp{\.exttomime{\fileext{#4}}}%
208   \_ifx\._tmp\_empty
209     \_opwarning{MIME type of '#4' unknown, using '\.defaultmimetype'}%
210     \_edef\tmp{\.defaultmimetype}%
211   \_fi
212   \sxddef{filemime:#3}{\tmp}%
213   \cs{filedef:#1}{#3}{#4}%
214 }
215 \_nspublic \filedef ;
```

Types “e”, “x”, “u” are predefined, anything else would essentially be a variant of these.

External file (“x”) is determined only by path.

pdfextra.opm

```
224 \sdef{filedef:x}#1#2{%
225   \slet{filespec:#1}{filename:#1}%
226 }
```

URL file (“u”) is determined by URL. Using all sorts of characters is allowed by using `\_detokenize`. This time it is necessary to create full *file specification* – a dictionary, where the “file system” is URL.

pdfextra.opm

```
234 \sdef{filedef:u}#1#2{%
235   \sdef{filespec:#1}{<</FS /URL /F (\_detokenize{#2})>>}%
236 }
```

Embedded files (“e”) are the most interesting ones. For further use (e.g. for displaying the embedded files as attachments) MIME type is required. It is saved in the stream as a `\Subtype`, encoded as a PDF name (e.g. `/video#2Fmp4`). The embedded file stream must be wrapped in a full *file specification*, which has the `/EF` (“embedded file”) entry. Also the friendly name is used for some purpose by PDF viewers, so it set in `/Desc` (description).

pdfextra.opm

```
248 \sdef{filedef:e}#1#2{%
249   \_edef\tmp{\cs{filemime:#1}}%
250   \_isfile{#2}\_iffalse
251     \_opwarning{file '#2' not found}%
```

```

252 \_fi
253 \_pdfobj stream
254   attr{/Type /EmbeddedFile /Subtype \_ea\mimetoname\_ea[\_tmp]}
255   file {#2}%
256 \_pdfrefobj\_pdflastobj
257 \.sxdef{filestream:#1}{\_the\_pdflastobj\_space 0 R}%
258 \_pdfobj {<</Type /Filespec
259   /F \.cs{filename:#1}
260   /Desc (#1)
261   /EF << /F \_the\_pdflastobj \_space 0 R >>%
262 >>}%
263 \_pdfrefobj\_pdflastobj
264 \.sxdef{filespec:#1}{\_the\_pdflastobj\_space 0 R}%
265 }

```

Now the less interesting part – determining the file names from paths and determining MIME types. The file name is the part after the last “/” (if any). The file extension is the part after last “.” (if any).

pdfextra.opm

```

273 \_def\.filename#1{\_ea\.filenameA#1\\.end}
274 \_def\.filenameA#1/#2{\_ifx\\.end#2#1\_else\_afterfi{\.filenameA#2}\_fi}
275
276 \_def\.fileext#1{\_ea\.fileextA#1\\.end}
277 \_def\.fileextA#1.#2{\_ifx\\.end#2#1\_else\_afterfi{\.fileextA#2}\_fi}

```

MIME type is determined from file extension (e.g. mp4 is “video/mp4”). For mapping of file extensions to MIME types we abuse T_EX’s hash table which gets populated with “known MIME types”. This necessarily means that the database is incomplete. Users can define their own additional mappings, or they can contribute generally useful ones to this package.

The default MIME type (used for unknown file extensions) is “application/octet-stream” – binary data.

The uninteresting MIME type database itself is at the very end (??).

pdfextra.opm

```

292 \_def\.mimetoname[#1/#2]{/#1\_csstring#2F#2}
293
294 \_def\.defaultmimetype{application/octet-stream}
295 \_def\.exttomime#1{\.trycs{mimetype:#1}{}}

```

Here we define an OpT_EX style “is-macro” that checks whether the file has already been defined – `\.isfiledefined{<name>}` (or `\iffalse`). The case where the file has not been defined using `\filedef` can be handled in a lot of ways. As a default we interpret `<name>` as path and try to embed it. Because the path from `<name>` is used as the “friendly name” the file will be embedded only once even when requested more times.

pdfextra.opm

```

307 \_def\.isfiledefined#1#2{\.isdefined{filespec:#1}\_iftrue\_else
308   \_afterfi{\.fileundefined{#1}}\_fi#2%
309 }
310
311 \_def\.fileundefined#1{\_isfile{#1}\_iftrue\.filedef/e[#1]{#1}\_else
312   \_opwarning{file '#1' not found, ignored}\_ea\_unless\_fi
313 }
314
315 % strict requirement of preceeding `filedef` can be set like this:
316 %\_def\.fileundefined#1{\_opwarning{file '#1' is not defined, ignored}\_unless}

```

2.4 PDF actions

The core of interactivity in PDF are actions. They are all initially handled by `\pdfaction[<action spec>]`. `<action spec>` is a comma separated list of `<type>:<arguments>`. Leading spaces in the elements of the list are ignored using undelimited-delimited argument pair trick.

An invocation could look like this:

```

\pdfaction[
  js:{app.alert("Yay JavaScript, going to page 5");},
  ilink:pg:5,
  transition:Wipe,
]

```

This is why we have to be very careful when loading the contents between `[]` to arguments. In particular, we can't split immediately using `[#1:#2]`, because this would discard the braces guarding the comma in the JavaScript code. However we also need to find out the *type* of action which is taken as a type of the first action (`js` in this case). `\pdfactiontype[⟨action spec⟩]` does this – we don't mind that there the braces are lost.

`\pdfaction` processes the list, to create a chain of actions using `/Next` field. The handling of each action type is up to macro `\pdfextra_⟨type⟩action`, which receives `[⟨type⟩:⟨arguments⟩]`. Because of this a single type handler can handle multiple different actions, as is the case with `\ilinkaction` which is the fallback for unknown action types.

pdfextra.opm

```

352 \_def\pdfaction[#1:#2]{\pdfactionA#1#2,\stop\end}
353 \_def\pdfactionA#1,#2#3\end{%
354   <<%
355   \pdfactionB[#1]%
356   % next action
357   \_ifx\stop#3\_else\_space
358   /Next \_afterfi{\pdfactionA#2#3\end} % intentional space
359   \_fi
360   >>
361 }
362 \_def\pdfactionB[#1:#2]{\trycs{#1action}{\_ea\ilinkaction}[#1:#2]}
363 \_nspublic \pdfaction ;
364
365 \_def\pdfactiontype[#1:#2]{#1}

```

2.4.1 Additional actions

Some PDF objects, like pages and some annotations, can also have “additional actions”. These are actions which will be executed when an event happens – like page getting opened for `/O` action in page's additional actions or `/PO` in annotation's additional actions. For constructing these additional actions we define a helper macro `\pdfaactions`. The use is as something follows:

```
/AA << \pdfaactions{ {O} {⟨action spec 1⟩} {C} {⟨action spec 2⟩} } >>
```

To produce something this:

```
/AA << /O <<⟨action 1⟩>> /C <<⟨action 2⟩>> >>
```

pdfextra.opm

```

384 \_def\pdfaactions#1{<<\pdfaactionsA #1\end\end>>}
385 \_def\pdfaactionsA#1#2{\_ifx\end#1\_else /#1 \_ea\pdfaction\_ea[#2]\_ea\pdfaactionsA\_fi}

```

2.4.2 Link annotations

The main use of actions – annotations of `/Subtype /Link`. Annotation of this type creates an active rectangular area on the page that executes a PDF action (or chain of them in the general case). `\hlink[⟨action spec⟩]⟨text⟩` is macro that typesets `⟨text⟩` and makes area occupied by it active according to `⟨action spec⟩`. All action types are supported, the mechanism is completely generic.

The `\pdfstartlink`/`\pdfendlink` primitives are used to denote the part of the page where `⟨text⟩` appears as active. LuaTeX will then handle even the situations where `⟨text⟩` gets broken across multiple lines (by creating multiple rectangular annotations to cover all `\hboxes`).

pdfextra.opm

```

403 \_def\hlink[#1]#2{\_bgroup\_def\#{\_csstring\#}%
404   \_edef\type{\pdfactiontype[#1]}%
405   \_quitvmode\pdfstartlink \linkdimens
406   attr{\pdfborder{\type}}%
407   user{/Subtype /Link /A \pdfaction[#1]}\_relax
408   \_localcolor\linkcolor{\type}#2\pdfendlink\_egroup
409 }
410
411 \_nspublic \hlink ;

```

Use `\hlink` as the backing command for OpTeX's “higher level” linking commands (`\ilink` and `\ulink`).

The lower level ones (`\xlink` and its predecessor `\link` actually have completely different semantics with regards to color, so we keep them as they are.

```

422 \protected\def\ilink[#1]#2{\.hlink[#1]{#2}}
423 \protected\def\ulink[#1]#2{{\_escapechar=-1 \_ea}\_expanded
424   {\_noexpand\.hlink[url:\_detokenize{#1}]{#2}}
425 }
426 \_public \link \ilink \ulink ;
427
428 %\protected\def\_link[#1]#2#3{\_hlink[#1]{#3}}
429 %\protected\def\_xlink#1#2#3#4{\_hlink[#1:#2]{#4}}

```

Two customizations of \hlinks are possible:

- Dimensions of rectangular areas created by \pdfstartlink/\pdfendlink. This is done using `\.linkdimens` (analogous to OpTeX’s \linkdimens). Dimensions that are unset are taken from the respective \hboxes. `\lininglinks` sets the dimensions for running text – it covers all space of a line using \baselineskip. `\nolininglinks` sets no dimensions, this is useful for buttons, that may have larger height/depth than a line.
- The color is determined from the type of link (that is, the first action in $\langle action\ spec \rangle$) by checking $_ \langle type \rangle linkcolor$ (compatible with OpTeX). As a fallback $_ ilinkcolor$ is used (set by OpTeX’s \hyperlinks) for all links except for URLs, where $_ elinkcolor$ is used instead. If even these fallback colors are not defined (\hyperlinks isn’t used), then the most generic $_ linkcolor$ will be taken or no color will be set.

```

450 \_def\.lininglinks{%
451   \_def\.linkdimens{height.75\_baselineskip depth.25\_baselineskip}%
452 }
453 \_def\.nolininglinks{\_def\.linkdimens{}}
454 \.lininglinks
455
456 \_nspublic \lininglinks \nolininglinks ;
457
458 \_def\.linkcolor#1{\_trycs{#1linkcolor}\_trycs{\_ilinkcolor}\_trycs{\_linkcolor}\_trycs{}}
459
460 % \_urllinkcolor = \_elinkcolor with fallbacks
461 \_def\_urllinkcolor{\.linkcolor{e}}

```

2.4.3 Open action

The document itself has one action defined in the document catalog. It is called /OpenAction. We allow the user to set it using the familiar $\langle action\ spec \rangle$ syntax with the command `\openaction[ $\langle action\ spec \rangle$ ]`.

Internally we could directly set it by appending to the catalog using the primitive \pdfcatalog, but LuaTeX (pdfTeX really) allows setting the action with special syntax. This has the benefit that it is not allowed to set the action more than once.

```

475 \_def\.openaction[#1]{\_pdfcatalog{} openaction user{\.pdfaction[#1]}\_relax}
476 \_nspublic \openaction ;

```

2.4.4 Jump actions

These are the most typical actions. Even LuaTeX itself handles them, although we don’t use the possibility for maintaining generality. There are a few types of jump actions:

- /GoTo actions are the classic internal links to named destinations in the PDF file (created by \pdfdest primitive or OpTeX’s \dest). The destination names include also the type of internal link (e.g. ref:section1). They are handled by `\.ilinkaction[ $\langle type \rangle$ : $\langle name \rangle$ ]`.
- /URI actions which are in most cases used as “goto URL” actions. These are not that useful directly, because special characters should be handled before this actions is used (like with \url). The low level use is `\.urlaction[url: $\langle url \rangle$ ]`.
- “Goto remote” actions, which can jump to a destination in another PDF file – either determined by name, or by page number. The external files are expected to be defined by \filedef (but not the embedded variant). The use is either `\.extrefaction[extref: $\langle name \rangle$ : $\langle named\ destination \rangle$ ]` for links to named destination or `\.extpgrefaction[extpgref: $\langle name \rangle$ : $\langle page\ number \rangle$ ]` for page destinations. Customization is possible with `\.extrefextra`, by default opening in a new windows is requested.

```

504 \_def\.\linkaction[#1:#2]{/S /GoTo /D (#1:#2)}
505
506 \_def\.\urlaction[#1:#2]{/S /URI /URI (#2)}
507
508 \_def\.\extrefaction[#1:#2:#3]{/S /GoToR
509 /F \.cs{filespec:#2}
510 /D (#3)
511 \.extrefextra
512 }
513 \_def\.\extpgrefaction[#1:#2:#3]{/S /GoToR
514 /F \.cs{filespec:#2}
515 /D [\_the\_numexpr#3-1\_relax\_space /Fit]
516 \.extrefextra
517 }
518
519 \_def\.\extrefextra{/NewWindow true}

```

Transition action is not really a jump action in of itself, but is only useful when chained after jump actions, so we define it here. Transitions (as page attributes) are handled more thoroughly in section ??.

The use would look something like:

`\.transitionaction[transition:<animation type>:<duration>:<raw PDF attributes>]`, where all fields omitted from right take the default values.

```

533 \_def\.\transitionaction[#1:#2]{/S /Trans \.attroempty{Trans}{\.maketrans[#2]}}

```

2.4.5 Named actions

User can request arbitrary “named” action with `\.namedaction[named:<name>]`. See user documentation for details.

```

542 \_def\.\namedaction[#1:#2]{/S /Named /N /#2}

```

2.4.6 JavaScript actions

JavaScript actions have two forms, either `\.jsaction[js:<name>]` or `\.jsaction[js:<script>]`. The first variant uses contents of `\filedef`’d `<name>`, the second one uses `<script>` directly. There is no special catcode handling.

```

554 \_def\.\jsaction[#1:#2]{/S /JavaScript
555 /JS \_ifcnsname _pdfextra_filestream:#2\_endcnsname \_lastnamedcs \_else
556 (#2)
557 \_fi
558 }

```

2.5 Page attributes

PDF represents pages as dictionaries. The dictionaries get generated by LuaTeX, which fills in some attributes *attributes* (like `/Content` with contents of the page and `/Annots` with array of annotations). We can add more using `\pdfpageattr` primitive token list register. While not that many are generally useful, there are a few interesting ones. For example transitions can be set using page attributes, or we might want to set additional actions (`/AA`) to listen for page events.

While the so called “page objects” are in a tree structure (for fast lookup), only the leaves are real “pages”. PDF allows some attributes to be inherited from parent page objects, but not all of them and certainly not those we are interested in.

The mechanism introduced in this section is optional, because it takes complete control over `\pdfpageattr`. It gets activated when `\initpageattributes` is first used (which happens automatically for some functionality exposed by this package), but may be activated by the user for any other purpose. Only attributes listed in `\pageattributes` are processed.

We set the attributes anew for each page, by hooking into OpTeX’s `\_begoutput`. Because `\pdfpageattr` token list doesn’t get expanded before written out to PDF, we expand it using the assignment in `\edef` trick. The token list gets expanded, but the assignment is not made until it reaches main processor when the temporary control sequence gets expanded.

```

590 % pdfpagattr managment (default for all pages vs current page override)
591 \_def\pageattributes{{Trans}{Dur}{Rotate}{AA}}
592 \_def\initpageattributes{%
593   % add hook for setting primitive \pdfpageattr
594   \_addto\_begoutput{\_edef\tmp{\_pdfpageattr={\pageattributes}}\tmp}%
595   % no need to do this twice
596   \_let\initpageattributes=\_relax
597 }
598 \_nspublic \pageattributes \initpageattributes ;

```

The user interface we want to expose has two parts:

- setting the page attribute for just this one page (`\pdfcurrentpageattr`),
- setting the default attribute (used when current page value is not set) (`\pdfdefaultpageattr`).

The first one of course brings in the typical T_EX problem of knowing the page where something occurs. As always, the page number contained in `\pageno` during processing of said content may of course not actually be the number of the page where the content ends up! Hence, we need to note the page number with a delayed write, using `\setpageof` and later `\pageof`. The different settings of page attributes should have distinct names, we use the `\pageattrcount` counter for this.

```

617 \_newcount\pageattrcount
618 \_def\pdfcurrentpageattr#1#2{\_initpageattributes
619   \_incr\pageattrcount
620   \_setpageof{pageattr:\_the\pageattrcount}%
621   \_sxddef{pdfpgattr:\_pageof{pageattr:\_the\pageattrcount}:#1}{#2}%
622 }
623 \_def\pdfdefaultpageattr#1#2{\_initpageattributes
624   \_sxddef{pdfpgattr:#1}{#2}%
625 }

```

Finally, the macro `\pdfpageattributes` takes care of setting generating the contents of `\pdfpageattr`. For each attribute in `\pageattributes` it first checks its current page value, only then the default value. If neither is set, nothing is added.

```

634 \_def\pdfpageattributes{\_ea\pdfpageattributesA\pageattributes\end}
635 \_def\pdfpageattributesA#1{\_ifx\end#1\_else
636   % use current page override or "default"
637   % don't emit anything if the value is empty
638   \_attroempty{#1}{%
639     \_trycs{pdfpgattr:\_the\_pageno:#1}{\_trycs{pdfpgattr:#1}{}}%
640   }%
641   \_ea\pdfpageattributesA\_fi
642 }

```

Each attributes then has two switches for the respective default and current values. For defining a few of them a helper is introduced:

`\pdfpageattributesetters` *<attribute>* *<default setter>* *<current setter>* *{<value>}*,
 where *<attribute>* is name of the attribute without the slash (e.g. `MediaBox`), the two control sequences name the future user setters, which will take single argument in brackets (e.g. `\mediabox` and `\thismediabox`) and the *<value>* can use the argument.

```

656 \_def\pdfpageattributesetters#1 #2#3#4{%
657   \_sdef{\_csstring#2}[##1]{\pdfdefaultpageattr{#1}{#4}}%
658   \_sdef{\_csstring#3}[##1]{\pdfcurrentpageattr{#1}{#4}}%
659   \_nspublic #2 #3 ;
660 }

```

Some of the useful attributes are `/Rotate`, which rotates the pages visually (can be set with `\defaultpagerotate` and `\pagerotate`), and the additional actions (`/AA`, see section 2.4.1, set using `\defaultpageactions` `\pageactions`).

```

669 \pdfpageattributesetters Rotate \defaultpagerotate \pagerotate {#1}
670
671 \pdfpageattributesetters AA \defaultpageactions \pageactions {\pdfaactions{#1}}

```

2.5.1 Transitions, page durations

There are predefined types of transitions, like `/Wipe`, `/Box`, `/Split`, etc. Most have other customizable attributes – usually directions set in different ways depending on the animation type at hand, but the most important attribute is the duration of the animation. Parsing friendly user notation in the form of `[<animation type>:<duration>:<other raw attributes>]`, where fields from the right may be omitted to produce the default value, is handled by `\.maketrans`. This macro is also used by transition actions (see 2.4.4). The defaults are simply those defined by PDF standard (no transition, 1 second duration and the respective default directions).

```
690 \_def\.maketrans[#1]{\.maketransA#1::\.end}  
691 \_def\.maketransA#1:#2:#3:#4\.end{%  
692   \.emptyor{#1}{<</S /\.nonempty \.attrorempty{D}{#2} #3>>}  
693 }
```

pdfextra.opm

The attribute setters for transitions (`\transitions`, `\transition`) are a simple wrappers. Similiar is the setting of page duration in seconds after which PDF viewer automatically advances to the next page (`\defaultpageduration`, `\pageduration`).

```
702 \.pdfpageattributesetters Trans \transitions \transition {\.maketrans[#1]}  
703  
704 \.pdfpageattributesetters Dur \defaultpageduration \pageduration {#1}
```

pdfextra.opm

2.6 Name trees – attachments and document level JavaScript

These don't have any last place to be in, so they are documented separately, here. Attaching files using `/FileAttachment` annotations:

1. is intended more towards viewers of the document for extra additions and
2. doesn't work in the viewers as well as one would like.

That is why instead embed files using normal `\filedef` and then allow them to be added to the document level `/EmbeddedFiles` entry, which means they will be shown in the user interface by PDF viewers. `/EmbeddedFiles` is a document level name tree (contained inside `/Names` entry of `/Catalog`) that maps names of files to their objects. Although we simplify matters by constructing more of an array.

What works very similiarly is document level JavaScript. It is a name tree within `/JavaScript` field. It maps names of JavaScript actions to their object numbers. The names aren't very useful, but the actions have their purpose. They are executed in turn after the document is opened. Hence they can be used to predefine JavaScript functions in the global context, to be used later within actions explicitly activated by the user.

The user level commands are `\attach[<name>]` (to attach a previously `\filedef`'d name with fallback to embedding now if it is a valid path) and `\dljavascript[<name>]{<script>}` (adds action that executes `<script>` after document is opened, `<name>` is more or less meaningless).

Internally the commands construct lists of what ends up in the resulting name array, i.e. pairs `(<name>)_<object_number>_O_R_`. Intermediate macros `\.embeddedfiles` and `\.dljavascripts` are used for this.

In the case of file attachments, nothing happens if file is defined and not found by the fallback.

```
745 % file attachment  
746 \_def\.embeddedfiles{  
747 \_def\.attach[#1]{\.isfiledefined{#1}\_iftrue  
748   \.xaddto\.embeddedfiles{(#1) \.cs{filespec:#1} }\_fi  
749 }  
750 \_nspublic \attach ;  
751  
752 \_def\.dljavascripts{  
753 \_def\.dljavascript[#1]#2{%  
754   \_immediate\_pdfobj{<< \.jsaction[js:{#2}] >>}%  
755   \.xaddto\.dljavascripts{(#1) \_the\_pdflastobj \_space 0 R }%  
756 }  
757 \_nspublic \dljavascript ;
```

pdfextra.opm

Renditions (see ??) also need their name tree. This package mostly doesn't play well with Unicode file-names, that is why they are forbidden. However, Renditions that are accessed from JavaScript have to be

named/present in a /Renditions name tree, with the names encoded in the PDF encoding (UTF-16BE). The names and object references are collected in `\.renditions`. Unicode encoding is hacked with `\.pdfstringtounicode`.

```
769 \_def\.pdfstringstrip(#1){#1}%
770 \_def\.pdfstringtounicode#1#2{%
771   \_ea\_pdfundef\_ea#1\_ea{\_ea\.pdfstringstrip\_expanded{\.cs{filename:#2}}}%
772 }
773 \_def\.renditions{}
```

pdfextra.opm

Object creation, which is common to all name trees, is handled by

`\.makenamearray`*<name tree name>**<name tree content>*.

It doesn't do anything for empty lists, to not bloat PDF files when this mechanism isn't used.

```
782 \_def\.makenamearray#1#2{\_ifx#2\_empty\_else
783   \_immediate\_pdfobj {<< /Names [ #2 ] >>}%
784   \_pdfnames{/#1 \_the\_pdflastobj \_space 0 R }\_fi
785 }
```

pdfextra.opm

The lists themselves can only be written out to the PDF file at the very end of the run. We use OpTeX's `\_byehook`, which is run in `\_bye`. But `\bye` itself may be predefined by the user, for example when using some of the OpTeX tricks. We just hope that the user keeps `\_byehook`.

```
794 \_addto\_byehook{%
795   \.makenamearray{EmbeddedFiles}\.embeddedfiles
796   \.makenamearray{JavaScript}\.dljavascripts
797   \.makenamearray{Renditions}\.renditions
798 }
```

pdfextra.opm

2.7 Viewer preferences

There are a few customizations of display (and other preferences of PDF viewers) possible in the document catalog or its subdictionary /ViewerPreferences. Most are not that useful. The interesting ones are implemented by `\fullscreen`, `\showoutlines`, `\showattached`. They all set the page mode using `\.setpagemode`. We don't handle respecting the last setting (using `\_byehook`). To prevent invalid PDF files, we set `\.setpagemode` to `\_relax` after use.

```
812 \_def\.setpagemode#1{\_pdfcatalog{/PageMode /#1}\_glet\.setpagemode=\_relax}
813
814 \_def\.fullscreen{\.setpagemode{FullScreen}}
815 \_def\.showoutlines{\.setpagemode{UseOutlines}}
816 \_def\.showattached{\.setpagemode{UseAttachments}}
817
818 \_nspublic \fullscreen \showoutlines \showattached ;
```

pdfextra.opm

Only the setting of duplex / double sided printing and display is in the nested dictionary. It is handled by `\duplexdisplay`. The simplistic version does not handle more attributes in /ViewerPreferences. We also set the meaning to `\_relax` to prevent more (erroneous) uses.

```
827 \_def\.duplexdisplay{\_pdfcatalog{%
828   /PageLayout /TwoPageRight
829   /ViewerPreferences <<
830     /Duplex /DuplexFlipLongEdge
831   >>}%
832   \_glet\.duplexdisplay=\_relax
833 }
834
835 \_def\.duplexdisplay{\.duplexdisplay}
```

pdfextra.opm

2.8 Multimedia

PDF essentially allows insertion of different types of multimedia:

- images,
- audio/video,
- 3D art.

The first is pretty standard and handled normally by the engine (LuaTeX). Others are possible, but have to be done manually according to one of the mechanisms specified by PDF standard:

- Sounds (audio only),
- Movies (video and/or audio),
- Renditions (video and/or audio),
- 3D annotations (3D art),
- Rich Media (video and/or audio, 3D art)

Sadly all these mechanisms are badly flawed, each in different ways. At least we try to use the one that works in the viewers.

For audio/video “Movies” are the simplest mechanism, but they have been deprecated in PDF 2.0 and no longer work in Acrobat/Foxit (same for “Sounds”).

“Renditions” are complicated, partly dependant on JavaScript, but at least supported by Acrobat, Foxit, Evince and Okular.

“Rich media” annotations were designed for Flash. This use case is no longer possible today, but the obscurities remain. They are unnecessarily complicated, but can be used without Flash too. Although the result is very plain for audio/video – no controls can be displayed and there are no associated actions.

“3D annotations” are reasonably simple, but also flawed. They cannot reuse embedded file as a source for 3D data. Hence it is better and more consistent to use Rich Media for 3D annotations. It even has additional benefits, like the possibility of using multiple initialization scripts.

In the end, this package exposes two user commands corresponding to two mechanisms – first are Renditions (`\render`) for audio/video that works in most browsers and Rich Media (`\RM`) mainly for 3D art, but also for audio/video with limited possibilities.

Both mechanisms have an annotation at their core. Annotations is essentially a rectangular area on page. The area corresponds to where the multimediuum will show up. After activating the area somehow (by user click, or action) the multimediuum will start playing. Before annotations the rectangular area will show something that is called “normal appearance”. This appearance is of type form XObject. Those are really similiar to pages – they have dimensions, contents made up of PDF graphics operators, ..., but they are reusable. Not that useful for annotations where we will need the form only once, but nice anyways. pdfTeX has primitives for creating them – `\pdfxform` and friends. They essentially do the same code like `\shipout` does, but instead of page, they make this reusable object. One can then either use this reusable object in another page/form, but we will indirectly refer to it for the appearance.

Important aspect of annotations is that they are really only rectangular areas on the page, but they are not really part of the page. They sort of sit on another level and are not influenced by PDF graphic operators which make the page. In pdfTeX annotations are handled by *whatsit* nodes. While most nodes map to known primitive TeX concepts (like typeset characters, boxes, rules, etc.) Whatsits are essentially commands for TeX that are delayed until page is being shipped out (written to PDF file). `\write`, `\special`, and most pdfTeX commands create whatsits. For annotationos this is important, because this means that the engine only stores the information about annotation that we specify, but creates it at due time, when it should be written to PDF.

Because whatsits are essentially dimensionless and we want it to be a part of normal TeX typesetting material we create the annotation (whatsit) in `\hbox`. This box will be otherwise empty, because the apperance of the rectangular area is determined by the normal appearance field (`/N` in `/AP`). We set the dimensions of the box to the dimensions of normal appearance. Everything will line up nicely, because when processed, the annotation will take dimensions from the box.

All of these concepts are implemented in:

`\.boxedannot` [*<type>* : *<name>*] {*<appearance>*} {*<special text>*} {*<annotation attributes>*}

<type> is used to determine the annotation border (same principle as with Link annotations, section 2.4.2), *<name>* will be used as the annotation name (`/NM`), *<special text>* is used for influencing the `\pdfannot` primitive, and *<annotation attributes>* will become the body of the annotation.

pdfextra.opm

```

928 \_def\.boxedannot[#1:#2]#3#4#5{%
929   \_setbox0=\_hbox{#3}\_setbox2=\_null
930   \_ht2=\_ht0 \_wd2=\_wd0 \_dp2=\_dp0
931   \_preshipout0 \_box0
932   \_immediate\_pdfxform0
933   % box with annotation both stretching to dimensions of appearance
934   \_hbox{\.setpageof{#1:#2}%
935     \_pdfannot #4 {#5

```

```

936      /AP <</N \_the\_pdflastxform \_space 0 R>>
937      \_pdfborder{#1}
938      /NM (#2)
939      /Contents (#1 '#2')
940  }%
941  \_copy2
942  }%
943 }

```

There is another weird thing common to both multimedia mechanisms – the redefinition of `\.name`. It is initially set by `\.secondoptdef` to `<name>`, but may be redefined by user supplied `name` key-value parameter. This should be used when there are multiple uses of the same content. Otherwise samely named annotations would be indistinguishable both for PDF viewer and our handling of actions (which would all refer only to the first instance).

To somewhat overcome this, trying to use the same `<name>` (within the same type of annotaiton) will use dummy name from `\.unnamedannotcount` (for uniqueness). This means that `<name>` will always refer to the first instance. `\.redefinename` handles this.

pdfextra.opm

```

960 \_newcount\.unnamedannotcount
961 \_def\.redefinename#1{%
962   \_isdefined{#1:\.name}\_iftrue
963     \_incr\.unnamedannotcount
964     \_edef\.name{\_the\.unnamedannotcount}%
965   \_else
966     \_edef\.name{\_kv{name}}%
967   \_fi
968 }

```

2.8.1 Renditions (audio/video)

There are three main types of PDF objects involved in the Renditions (“Multimedia”) mechanism:

- Screen annotations define the area for playing multimedia.
- Rendition objects define the multimedia to play.
- Rendition actions associate Rendition objects with Screen annotations.

You can theoretically arbitrarily mix and match rendition objects and screen annotations by invoking different actions. In practice Evince and Okular do really simplistic parsing and don’t fully support the actions fully. But by keeping it simple it is possible to make it work almost the same in all viewers that support renditions.

Different sources of audio/video should be possible. In fact all three file specifications (embedded files, files specified by URL/path) could work. Again in practice embedded file is the safest bet, that works in all viewers that support renditions.

The user facing command is:

`\render[<name>][<optional key-value paramers>]{<horizontal material>}`

`<name>` is the friendly name set using `\filedef` or file path if `<name>` isn’t `\filedefd` and is to be embedded. The key-value parameters in brackets can be entirely omitted. They can influence the playback (except for `controls` most are not well supported). Default values are taken from `\.renderdefaults`.

`\render` doesn’t do anything (except print warning) if file `<name>` isn’t defined and `<name>` isn’t a path to file that can be embedded.

The first PDF object it defines is Rendition, which specifies information about the multimediu (name, file specification, MIME type and options from key-value parameters). Some of the fields are in /BE (“best effort”) dictionaries. This is due to the very general design of Renditions, which theoretically allows the PDF viewer to choose from multiple Renditions if they know they can’t support some of the requested features. But that is not much useful in practice, so we just don’t complicate it.

Next defined object is Screen annotation, which complicates thing by requiring (/P) reference to the page where the annotation is (handled by `\setpageof` and `\pageof` pair). Important field is /A which specifies actions that shall be executed when the screen area is clicked. We let the user change the action, but the sensible default of starting to play the multimediu is used (and this is the only thing that works in some viewers anyways). Additional actions /AA may be used to react to events like mouse over or page open/close – the most probable use case is autoplay on page open, for which shortcut of `\renditionautoplay` is defined.

The code is slightly complicated by the fact, that actions need to reference the Rendition and Screen objects. In the case of the action contained in Screen annotation this essentially involves a self reference. Hence it is needed to first reserve an object number and later use it for the annotation. Because the object numbers may also be needed by actions defined later, we need to save them to `\_pdfextra_rendition:<name>` and `\_pdfextra_screen:<name>` respectively, but also define aliases with empty names, so users can easily reference the latest rendition.

pdfextra.opm

```

1034 \.secondoptdef\render#1{\.isfiledefined{\.name}\_iftrue\_bgroup
1035   \_ea\_readkv\_ea{\_ea\renderdefaults\_ea,\_the\_opt}%
1036   \.colortorgbdef\bgcolor{\_kv{background}}}%
1037   % rendition object ("media specifaction")
1038   \_pdfstringtounicode\_uname\.name
1039   \_pdfobj {<</Type /Rendition
1040     /S /MR
1041     /N \.uname
1042     /C <</Type /MediaClip
1043       /S /MCD % subtype MediaClipData
1044       /D \.cs{filespec:\.name}
1045       /CT (\.cs{filemime:\.name})
1046       /P << /TF (TEMPALWAYS) >> % allow creating temporary files
1047     >>
1048     /P <</Type /MediaPlayParams
1049       /BE << /C \_kv{controls} /V \_kv{volume} /RC \_kv{repeat} >>
1050     >>
1051     /SP <</Type /MediaScreenParams
1052       /BE << /O \_kv{opacity} /B [\bgcolor] >>
1053     >>
1054   >>}\_pdfrefobj\_pdflastobj
1055   \.xaddto\renditions{\.uname \_the\_pdflastobj \_space 0 R}%
1056   \.redefinename{rendition}%
1057   \.sxddef{rendition:\.name}{\_the\_pdflastobj}%
1058   % screen annotation ("screen space allocation")
1059   \_pdfannot reserveobjnum% "self" reference will be needed inside screen annot.
1060   \.sxddef{screen:\.name}{\_the\_pdflastannot}%
1061   % aliases to latest rendition/screen with empty name
1062   \_global\slet{rendition:}{rendition:\.name}%
1063   \_global\slet{screen:}{screen:\.name}%
1064   \_edef.action{\_kv{action}}\_edef.aactions{\_kv{aactions}}%
1065   \.boxedannot[rendition:\.name]{#1}{useobjnum\_the\_pdflastannot}{%
1066     /Subtype /Screen
1067     % reference to page of the rendition (\setpageof done by \.boxedannot)
1068     % the spaces are weird, but \pdfpageref eats them
1069     /P \_pdfpageref\.pageof{rendition:\.name} \_space 0 R
1070     /A \_ea\.pdfaction\_ea{\.action}
1071     /AA \_ea\.pdfaactions\_ea{\.aactions}
1072   }%
1073   \_egroup\_fi
1074 }
1075 \_nspublic \render ;

```

Here are the defaults used for `\render` – `\.renderdefaults`. Users can redefine them all together or override as needed with key-value parameters. The defaults correspond to values specified by PDF standard. Other values may not be respected by all viewers.

pdfextra.opm

```

1084 \_def\.renderdefaults{%
1085   name=\.name,
1086   controls=false,
1087   volume=100,
1088   repeat=1,
1089   opacity=1.0,
1090   background=1 1 1,
1091   action=rendition::play,
1092   aactions={},
1093 }

```

Most probable use of additional actions is to start auto-start playing of the multimediuum. For this purpose `\renditionautoplay` is defined as a shorthand for action to play the lastly defined rendition on page visible event.

```

1101 \_def\renditionautoplay{{PV}}{rendition::play}}
1102 \_nspublic \renditionautoplay ;

```

Rendition actions

Rendition actions unfortunately use cryptic symbolic numbers (0, 1, 2 and 3) for actions that could be called **play**, **stop**, **pause** and **resume** respectively. Except for these predefined actions (that use /OP) running of JavaScript is possible using /JS (*<script>*) with potential fallback to /OP. This is dangerous territory, because support of the right API in the viewer is very low. Although it is possible to define such action type by:

```
\.sdef{renditionaction:myaction}{/JS (app.alert("something useful");) /OP 0}
```

The use of rendition action is: `\.renditionaction[rendition:<name>:<action type>]`. Empty name refers to last rendition, so e.g. `\.renditionaction[rendition::pause]` is possible.

pdfextra.opm

```

1124 \.sdef{renditionaction:play}{/OP 0}
1125 \.sdef{renditionaction:stop}{/OP 1}
1126 \.sdef{renditionaction:pause}{/OP 2}
1127 \.sdef{renditionaction:resume}{/OP 3}
1128 \_def\renditionaction[#1:#2:#3]{/S /Rendition
1129   \.cs{renditionaction:#3}
1130   /R \.cs{rendition:#2} 0 R
1131   /AN \.cs{screen:#2} 0 R%
1132 }

```

2.8.2 Rich Media (3D/audio/video)

Some principles seen with Renditions (section 2.8.1) apply here too. But additionally we deal with 3D specifics and unfortunate Flash leftovers.

Unlike Renditions both page area and multimediuum specifaction are handled in a single annotation – the Rich Media annotation. The code is unfortunately obscured due to the weird requirements, but this is essentially what we are trying to create with `\RM`:

```

/Type /Annot
/Subtype /RichMedia
/RichMediaSettings <<
  /Activation <<
    /Condition /PV
    /Scripts [ 14 0 R ]
  >>
  /Deactivation << /Condition /XD >>
>>
/RichMediaContent <<
  /Assets << /Names [ (kladka.prc) 2 0 R (wireframe.js) 14 0 R ] >>
  /Configurations [ <<
    /Type /RichMediaConfiguration
    /Subtype /3D
    /Instances [ <<
      /Type /RichMediaInstance
      /Subtype /3D
      /Asset 2 0 R
    >> ]
  >> ]
>>

```

The activation/deactivation will be dealt with later. But we see that to insert a simple 3D file, we have to pack it inside a file specification (indirect reference to object 2 0 R), then in “instance”, inside a “configuration” inside “content”. As if it wasn’t enough the names (normally contained in the file specification) have to be specified again in **Assets** name tree that uselessly maps names to file specifications. Because this is a 3D Rich Media annotation there are other files at play – initialization scripts. These are specified in **/Scripts** and are executed in turn when the annotation is activated. Not shown is, that some “configurations” and “instances” actually have to be specified indirectly.

If it wasn't for Flash we could do with something like:

```
/Type /Annot
/Subtype /RichMedia
/Activation /PV
/Scripts [ 14 0 R ]
/Deactivation /XD
/Content 2 0 R
```

Which contains equivalent information. But unfortunately here we are...

pdfextra.opm

```
1194 \.secondoptdef\RM#1{\.isfiledefined{\.name}\_iftrue
1195   \_edef\_.tmp{\.cs{filemime:\.name}}}%
1196   \_edef\_.subtype{\_ea\_.mimetormsubtype\_ea[\_.tmp]\_space}%
1197   \_ifx\_.subtype\_space
1198     \_opwarning{unknown rich media type for '\.name', ignored}\_else
1199   \_bgroup
1200   \_ea\_readkv\_ea{\_ea\_.RMdefaults\_ea,\_the\_opt}%
1201   % Instance that has the media file as an asset
1202   \_pdfobj {<</Type /RichMediaInstance
1203     /Subtype /\_.subtype
1204     /Asset \.cs{filespec:\.name}
1205   >>}\_pdfrefobj\_pdflastobj
1206   % Configuration with one single instance (the above)
1207   \_pdfobj {<</Type /RichMediaConfiguration
1208     /Subtype /\_.subtype
1209     /Instances [ \_the\_pdflastobj \_space 0 R ]
1210   >>}\_pdfrefobj\_pdflastobj \_edef\_.configuration{\_the\_pdflastobj}%
1211   \_edef\_.names{\.cs{filename:\.name} \.cs{filespec:\.name} }% initial asset
1212   \.redefinename{rm}%
1213   \_def\_.scriptfilespecs{}%
1214   \_edef\_.views{\_kv{views}}\_edef\_.scripts{\_kv{scripts}}%
1215   \_ifx\_.views\_empty \_edef\_.views{\.name}\_fi
1216   \_ea\_.DDDscripts\_ea{\_.scripts}%
1217   % annotation in hbox
1218   \.boxedannot[rm:\.name]{#1}{}%
1219   /Subtype /RichMedia
1220   /RichMediaSettings <<
1221     /Activation <<
1222       /Condition \.cs{activation:\_kv{activation}}
1223       \.emptyor{\_.scriptfilespecs}{/Scripts [ \_.nonempty ]}
1224       /Presentation << /Toolbar \_kv{toolbar} \.RMPresentationextra >>
1225     >>
1226     /Deactivation << /Condition \.cs{deactivation:\_kv{deactivation}} >>
1227   >>
1228   /RichMediaContent <<
1229     /Assets << /Names [ \_.names ] >>
1230     /Configurations [ \_.configuration \_space 0 R ]
1231     \.emptyor{\_ea\_.DDDviews\_ea{\_.views}}{/Views [ \_.nonempty ]}
1232   >>
1233   }%
1234   \.sxdef{rm:\.name}{\_the\_pdflastannot}%
1235   \_global\_.slet{rm:}{rm:\.name}%
1236   \_egroup\_fi\_fi
1237 }
1238 \_nspublic \RM ;
```

The code is similar to `\render`, but we also ignore everything if we don't recognize the type of media (Video, Sound or 3D). For that we use a simple mapping from MIME types with `\.mimetormsubtype`. This means that although we aim Rich Media mostly for 3D art it may also be used for Video and Sound.

pdfextra.opm

```
1248 \_def\_.mimetormsubtype[#1/#2]{\.cs{rmtype:#1}}
1249
1250 \.sdef{rmtype:model}{3D}
1251 \.sdef{rmtype:video}{Video}
1252 \.sdef{rmtype:audio}{Sound}
```

Then we also need to construct the weird name “tree” (essentially an array in our case) and script array. `\.DDDscripts` and `\.DDDviews` do this. Name tree is accumulated in `\.names`, and starts with the media

file. After that each script is added to `\.names` and `\.scriptfilespecs`. The scripts are passed as a comma separated array. Ignoring initial spaces is done using delimited-delimited argument pair trick.

```
1263 \_def\DDDscripts#1{\DDDscriptsA#1,,,\end}
1264 \_def\DDDscriptsA#1#2,{\_ifx,#1\_ea\untilend\_else
1265   \isfiledefined{#1#2}\_iftrue%
1266     \addto\scriptfilespecs{\cs{filespec:#1#2} }%
1267     \addto\names{\cs{filename:#1#2} \cs{filespec:#1#2} }%
1268   \_fi
1269   \_ea\DDDscriptsA\_fi
1270 }
```

For 3D views we need to process yet another comma separated list, this time with `\DDDviews`. The result has to be separated by spaces and we also don't want to emit something if the specified view was invalid. Unfortunately this is expansion only context, so we can't issue a warning.

As a user convenience, before `\DDDview` is executed, view with the name of `\.name` is tried instead of empty view array. This means that for simple 3D art with one view, one can create view with the same name as the 3D object and not have to specify anything. We also take the name only after it is redefined from optional key-value parameters – this is so we can support even the case of e.g. `screw` 3D model used twice, once with `name=screw1`, another time with `name=screw2` (with the corresponding `screw1` and `screw2` views). This is probably less useful, but...

```
1288 \_def\DDDviews#1{\DDDviewsA#1,,,\end}
1289 \_def\DDDviewsA#1#2,{\_ifx,#1\_ea\untilend\_else
1290   \isdefined{3dview:#1#2}\_iftrue
1291     \_lastnamedcs\_space \_fi
1292   \_ea\DDDviewsA\_fi
1293 }
```

The activation/deactivation names are kind of cryptic, so we give them descriptive names. Default is explicit (de)activation. Instead of `/PV` (page visible) and `/PI` (page invisible) it would be possible to use “page open” and “page close”. These are slightly different in cases when more pages are shown on screen at once, because only one page is “open”, while multiple are “visible”.

```
1304 \sdef{activation:explicit}{/XA}
1305 \sdef{activation:auto}{/PV}
1306 \sdef{deactivation:explicit}{/XD}
1307 \sdef{deactivation:auto}{/PI}
```

Additional means of customization are here. `\RMdefaults` contains the default key-value parameters. `\RMpresentationextra` can be used to set more attributes in `/RichMediaPresentation` dictionary (although those are more specific and not generally useful).

```
1316 \_def\RMdefaults{%
1317   name=\.name,
1318   activation=explicit,
1319   deactivation=explicit,
1320   toolbar=true,
1321   views=,
1322   scripts=,
1323 }
1324 \_def\RMpresentationextra{}
```

For scripting using JavaScript actions one needs to access the 3D context of the 3D / Rich Media annotation. This requires the page number. We can't use `this.pageNum`, because the script strictly doesn't have to be on the same page. We use `\.pageof` (`\.setpageof` was done in `\.boxedannot`) to retrieve the page number in next run. Also PDF indexes page numbers from 0. `\DDDannot{<name>}` and `\DDDcontext{<name>}` allow this.

```
1335 \_def\DDDannot#1{%
1336   this.getAnnotRichMedia(\_the\_numexpr\.pageof{rm:#1}-1\_relax, '#1')%
1337 }
1338 \_def\DDDcontext#1{\DDDannot{#1}.context3D}
1339
1340 \_nspublic \DDDannot \DDDcontext ;
```

2.8.3 3D views

This is the interesting part about 3D art. They can have a set of predefined views – although a user may start from one, they can interactively change all the aspects by dragging with mouse or messing with the settings shown by right click menu.

There are several transformations that have to be done before it is possible to display 3D scene on a computer screen:

1. 3D transformation from the coordinate system of 3D artwork (“model”) to the “world coordinate system”.
2. 3D transformation from the world coordinate system to camera coordinate system.
3. projection to 2D (3D to 2D transformation).

When talking about PDF, positive x goes to the right, positive y up, and positive z “away” from us (“into the page”). This means we are working with a left handed coordinate system. In camera space, the camera sits at $(0, 0, 0)$ facing towards positive z with positive x and y going right and up respectively. Projection (one way or another) discards the z coordinate.

Although the transformations are not strictly linear, they are essentially done using multiplication by *transformation matrices*. The matrix for “model to world” (or “model”) transformation is part of the 3D art file and can’t be changed. However, we can make it up, because we can fully control the second transformation (“world to camera” or “view” transformation) – although we don’t specify the “world to camera” matrix but rather its inverse, the “camera to world” matrix ($/C2W$). This matrix has the 4×4 form, which also allows *linear transformation* and *translation*:

$$M_{c2w} = \begin{pmatrix} a & d & g & t_x \\ b & e & h & t_y \\ c & f & i & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Here we use the column major convention, which is also the order how we would write the matrix to PDF file, where it is an array of 12 elements:

$$/C2W [a \ b \ c \ d \ e \ f \ g \ h \ i \ t_x \ t_y \ t_z]$$

In the rendering pipeline everything is transformed from world coordinates to camera space coordinates. We can think about the process also in the other way. Using M_{c2w} we specify camera’s position and orientation in the world coordinate system. Due to how transformation using matrix multiplication works, the first column in the M_{c2w} matrix (vector $(a, b, c)^T$) specifies how “positive x direction” (“right”) from camera space ends up in world coordinate system. Similarly for $(d, e, f)^T$ being the image of positive y (“up”) and $(g, h, i)^T$ being the image of positive z (“forward”). The last column, $(t_x, t_y, t_z)^T$ represents translation from camera space to world. Translation of origin (camera position) will leave it in the point with coordinates (t_x, t_y, t_z) . Because of these associations with the intuitive meanings of x , y , z in camera space we also sometimes call the vectors in the first three columns of M_{c2w} “right”, “up” and “forward” and the last one “eye”:

$$\vec{R} = \begin{pmatrix} a \\ b \\ c \end{pmatrix}, \quad \vec{U} = \begin{pmatrix} d \\ e \\ f \end{pmatrix}, \quad \vec{F} = \begin{pmatrix} g \\ h \\ i \end{pmatrix}, \quad \vec{E} = \begin{pmatrix} t_x \\ t_y \\ t_z \end{pmatrix}.$$

We usually want $\vec{R}$, $\vec{U}$ and $\vec{F}$ to form an orthonormal set of vectors, i.e. all of unit length and each pair is orthogonal. The orthogonality will come from the way we calculate them, but the normality has to be ensured by normalizing the vectors after computing them, which will not be explicitly written out in the following text. $\vec{E}$ is a positional, not directional, vector and it’s length will be preserved.

Now we only need a convenient way to calculate all four vectors. A wide spread method is sometimes called “look at”. It essentially involves having two points: “eye” (E , position of the camera) and “target” (T , the point where the camera is pointing at). The camera position is already provided:

$$\vec{E} = E$$

From these two points alone we can easily calculate the forward vector, which corresponds to the direction of the camera:

$$\vec{F} = \overrightarrow{ET} = T - E$$

When we now imagine the point E and vector $\vec{F}$ looking towards T we can see that there is a degree of freedom – the camera can rotate about the forward vector. There is no other way than to arbitrarily choose either up or right vector. Usually we choose an arbitrary “global up” vector $\vec{U}_G$, which will influence the general direction of the final up vector. This is because we use it to calculate the right vector:

$$\vec{R} = \vec{U}_G \times \vec{F}$$

The cross product makes it so that:

1. $\vec{R}$ is perpendicular to $\vec{F}$
2. it is also perpendicular to global up vector ($\vec{U}_G$) which we used to get rid of remaining degree of freedom.

Now that we have two orthonormal vectors (with the normalization not being explicit) we can calculate the remaining up vector:

$$\vec{U} = \vec{F} \times \vec{R}$$

The last mysterious part about the calculation are the cross products. They are of course not commutative, so why e.g. $\vec{U}_G \times \vec{F}$ and not the other way around? This is because we have to preserve the relations these vectors had as directions of positive axes of the original camera space. There we had positive x going right, positive y up and positive z forward in a left handed coordinate system. This means that following holds (according the left hand rule):

$$\vec{R} = \vec{U} \times \vec{F}$$

$$\vec{U} = \vec{F} \times \vec{R}$$

$$\vec{F} = \vec{R} \times \vec{U}$$

The scheme has one flaw though. When the directions of global up vector $\vec{U}_G$ and forward vector $\vec{F}$ are linearly dependent the computed right vector will be $(0, 0, 0)$. Hence some handling of this special case is needed.

The “look at” method is essentially what is used in Alexander Grahn’s package `movie15`¹. Although the input aren’t two points, but rather a “center of orbit” point (COO , our “target”), “center of orbit to camera vector” ($\overrightarrow{C2C}$, default is $(0, -1, 0)$) and distance of camera from the center of orbit (ROO). The value used for the arbitrary “global up” vector is $(0, 0, 1)$. When forward vector is $(0, 0, z)$, then global up is chosen to be $(0, -1, 0)$ or $(0, 1, 0)$ to handle the “0 right vector” issue.

Because the `movie15` method of providing the parameters is used in essentially all packages that handle PDF 3D art (`movie15`, `media9`, `rmannot`, `ConTeXt`) we also follow the suite.

`\DDDview[⟨view name⟩][⟨key-value parameters⟩]` is the command for defining 3D views. These have to be saved into separate PDF objects anyways, using this interface we allow their reuse. If $\langle view name \rangle$ is same as $\langle name \rangle$ of `\RM` argument and no other views are specified $\langle view name \rangle$ view is automatically used (see `\RM` for details).

Key-value parameters are not optional this time, because rarely one suffices with default values – different 3D views are about customization. Handling of them is not as straightforward as before. We initially read the key-value parameters only to determine the `method` used for calculating the `/C2W` matrix. Then we reread key-value parameters again, this time with also with the default values for this particular method. Not that the general 3d views details are changed, but the methods themselves have key-value parameters of their own, and we support specifying them in this “flat” way.

Additionally we allow the different methods used to compute `/C2W` to not be expandable. Hence they are executed outside of expansion only context and are fully processed – the text they add to 3D view PDF object is temporarily stored in `\.viewparams`.

The rest is simply setting sensible defaults (or user overrides) for internal/external name of the view (`/IN` and `/XN`, one is used for scripting, one is shown by the PDF viewer), background color, rendering mode, and lighting. Cross sections and nodes are currently not supported, although users can hook in their own code using `\.DDDviewextra`, `\.DDDrendermodeextra` or `\.DDDprojectionextra`.

We have to be careful about setting rendering mode and lighting scheme, because they normally fall back to the values specified in 3D art file, which we can’t access, so better not set them to anything if they are empty.

¹ <https://www.ctan.org/pkg/movie15>

```

1516 \_def\DDDview[#1][#2]{\_bgroup
1517   \_readkv{\DDDviewdefaults,#2}%
1518   \_edef\TMP{\DDDviewdefaults,\cs{3dview:\_kv{method}:defaults}}%
1519   \_readkv{\TMP,#2}%
1520   \colorcmykdef\bgcolor{\_kv{background}}%
1521   \cs{3dview:\_kv{method}}% sets \viewparams (/MS, /C2W, /CO)
1522   \pdfobj {<</Type /3DView
1523     /XN (#1)
1524     /IN (#1)
1525     \viewparams % /MS, /C2W, /CO
1526     /P <<
1527       \cs{3dprojection:\_kv{projection}}
1528       \DDDprojectionextra
1529     >>
1530     /BG <<{/Type /3DBG
1531       /Subtype /SC
1532       /C [\bgcolor]
1533     >>
1534     \_emptyor{\_kv{rendermode}}{%
1535       /RM <<{/Type /3DRenderMode
1536         /Subtype /\_nonempty
1537         \DDDrendermodeextra >> }%
1538     \_emptyor{\_kv{lighting}}{%
1539       /LS <<{/Type /3DLightingScheme
1540         /Subtype /\_nonempty >> }%
1541     >>}%
1542     \pdfrefobj\_pdflastobj
1543     \_sxdef{3dview:#1}{\_the\_pdflastobj \_space 0 R}%
1544   \_egroup
1545 }
1546
1547 \_nspublic \DDDview ;

```

`\DDDviewdefaults` stores default key-value parameters for 3D views. They are mostly the PDF standard defaults or what movie15/media9 uses (for compatibility). `\DDDviewextra`, `\DDDrendermodeextra` or `\DDDprojectionextra` can be used by the users to hook themselves into 3D view object creation.

```

1558 \_def\DDDviewdefaults{
1559   projection=perspective,
1560   scale=1,
1561   ps=Min,
1562   FOV=30,
1563   background=1 1 1,
1564   rendermode=,
1565   lighting=,
1566   method=media9,
1567 }
1568 \_def\DDDprojectionextra{}
1569 \_def\DDDrendermodeextra{}
1570 \_def\DDDviewextra{}

```

There are two different projection methods:

- Orthographic: z coordinate is simply thrown away, `scale` is used for scaling the result. For technical parts where we want lines that are parallel stay parallel in the view.
- Perspective: is the way human eye sees. `FOV` can be used to set field of view. (`ps` parameter for additional scaling to fit width/height is also available, but the default is fine for casual users).

```

1584 \_sdef{3dprojection:ortho}{/Subtype /O /OS \_kv{scale}}
1585 \_sdef{3dprojection:perspective}{/Subtype /P /FOV \_kv{FOV} /PS /\_kv{ps}}

```

We offer the possibility of setting `/C2W` matrix and `/CO` (distance from camera to center of orbit) directly using `method>manual`.

```

1593 \_sdef{3dview>manual:defaults}{
1594   matrix=1 0 0 0 1 0 0 0 1 0 0 0 ,
1595   centeroforbit=0,
1596 }

```

```

1597 \.sdef{3dview>manual}{\_edef\.viewparams{
1598 /MS /M
1599 /C2W [\_kv{matrix}]
1600 /C0 \_kv{centeroforbit}
1601 }}

```

Another simple way of specifying camera position/orientation is to use a named setting of U3D file using a U3D path with method=u3d.

pdfextra.opm

```

1608 \.sdef{3dview:u3d:defaults}{
1609 u3dpath=,
1610 }
1611 \.sdef{3dview:u3d}{%
1612 \_pdfunidef\.tmp{\_kv{u3dpath}}%
1613 \_edef\.viewparams{
1614 /MS /U3D
1615 /U3DPath \.tmp \_space
1616 }}

```

The most advanced method of setting /C2W matrix and /C0 is method=media9. It is thoroughly explained above, the few differences are because the input values are not two points. Also for conciseness “x”, “y” and “z” are used instead of right, up and forward. The calculations are done in Lua, for simplicity.

We expect the user to supply the numbers in the form “1 2 3”, but in Lua we need them comma separated (“1, 2, 3”). **\.luatriplet** does this. Just in case the code is somehow adapted without ensuring that x and z are orthonormal, we normalize also y after the second cross product.

pdfextra.opm

```

1631 \_def\.luatriplet#1 #2 #3 {#1, #2, #3}
1632
1633 \.sdef{3dview:media9:defaults}{%
1634 roo=0,
1635 coo=0 0 0,
1636 c2c=0 -1 0,
1637 }
1638 \.sdef{3dview:media9}{\_edef\.viewparams{
1639 /MS /M
1640 /C2W [\_directlua{
1641 local function normalize(x, y, z)
1642     local len = math.sqrt(x*x + y*y + z*z)
1643     if len \csstring\-= 0 then return x/len, y/len, z/len else return 0, 0, 0 end
1644 end
1645 local function cross(ux, uy, uz, vx, vy, vz)
1646     return uy*vz - uz*vy, uz*vx - ux*vz, ux*vy - uy*vx
1647 end
1648 local function printmat(...)
1649     local arr = table.pack(...)
1650     for k, v in ipairs(arr) do
1651         arr[k] = string.format("\_pcent.6f", v)
1652     end
1653     tex.print(table.concat(arr, " "))
1654 end
1655
1656 local roo = \_kv{roo}
1657 local coo_x, coo_y, coo_z = \_ea\.luatriplet\_expanded{\_kv{coo}}
1658 local c2c_x, c2c_y, c2c_z = normalize(\_ea\.luatriplet\_expanded{\_kv{c2c}})
1659
1660 local eye_x, eye_y, eye_z = coo_x + c2c_x*roo, coo_y + c2c_y*roo, coo_z + c2c_z*roo
1661
1662 local z_x, z_y, z_z = -c2c_x, -c2c_y, -c2c_z
1663
1664 local up_x, up_y, up_z = 0, 0, 1
1665 if math.abs(z_x) + math.abs(z_y) < 0.0000001 then % z_x == 0 and z_y == 0
1666     if z_z < 0.0000001 then % z_z <= 0
1667         up_x, up_y, up_z = 0, 1, 0
1668     else
1669         up_x, up_y, up_z = 0, -1, 0
1670     end
1671 end
1672
1673 local x_x, x_y, x_z = normalize(cross(up_x, up_y, up_z, z_x, z_y, z_z))

```

```

1674 local y_x, y_y, y_z = normalize(cross(z_x, z_y, z_z, x_x, x_y, x_z))
1675
1676 local eye_x, eye_y, eye_z = coo_x - z_x*roo, coo_y - z_y*roo, coo_z - z_z*roo
1677
1678 printmat(x_x, x_y, x_z, y_x, y_y, y_z, z_x, z_y, z_z, eye_x, eye_y, eye_z)
1679 }]
1680 /C0 \_kv{roo}
1681 }}

```

Last, but not least, is an action for setting the 3D view of a 3D/RM annotation using an action. `\.goto3dviewaction`[goto3dview:<name>:<view>]. <name> is name of the annotation which will be influenced. <view> is passed directly to PDF. Therefore it can be either an index to the view array (starting at 0) or name of view in parentheses – “(<view name>)”.

pdfextra.opm

```

1691 \.sdef{goto3dviewaction}[#1:#2:#3]{/S /GoTo3DView
1692 /TA \.cs{rm:#2} 0 R
1693 /V #3
1694 }

```

2.9 MIME type database

This is the uninteresting MIME type database teased in section 2.3. Ideally this would only be a subset of what IANA defines at <https://www.iana.org/assignments/media-types/media-types.xhtml>. But there are additions like `model/u3d` and `model/prc`, which don’t seem to be official, yet. Other “unofficial” MIME types are taken from Mozilla’s “common” lists:

- https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/MIME_types.
- https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/MIME_types/Common_types

`\.mimetype{<extension>}{<MIME type>}` is a shortcut of mapping <extension> to <MIME type>.

pdfextra.opm

```

1716 \_def\.mimetype#1#2{\_sdef{pdfextra_mimetype:#1}{#2}}
1717
1718 \.mimetype{js}{application/javascript}
1719 \.mimetype{pdf}{application/pdf}
1720
1721 \.mimetype{prc}{model/prc}
1722 \.mimetype{u3d}{model/u3d}
1723
1724 \.mimetype{wav}{audio/x-wav}
1725 \.mimetype{mp3}{audio/mpeg}
1726 \.mimetype{opus}{audio/opus}
1727
1728 \.mimetype{avi}{video/x-msvideo}
1729 \.mimetype{mp4}{video/mp4}
1730 \.mimetype{webm}{video/webm}
1731
1732 \.mimetype{tex}{application/x-tex}
1733
1734
1735 % more or less legacy formats
1736 \.mimetype{au}{audio/basic}
1737 \.mimetype{aiff}{audio/x-aiff}
1738 \.mimetype{mov}{video/quicktime}
1739 \.mimetype{mpg}{video/mpeg}
1740 \.mimetype{wmv}{video/x-ms-wmv}
1741
1742 \_endnamespace

```